

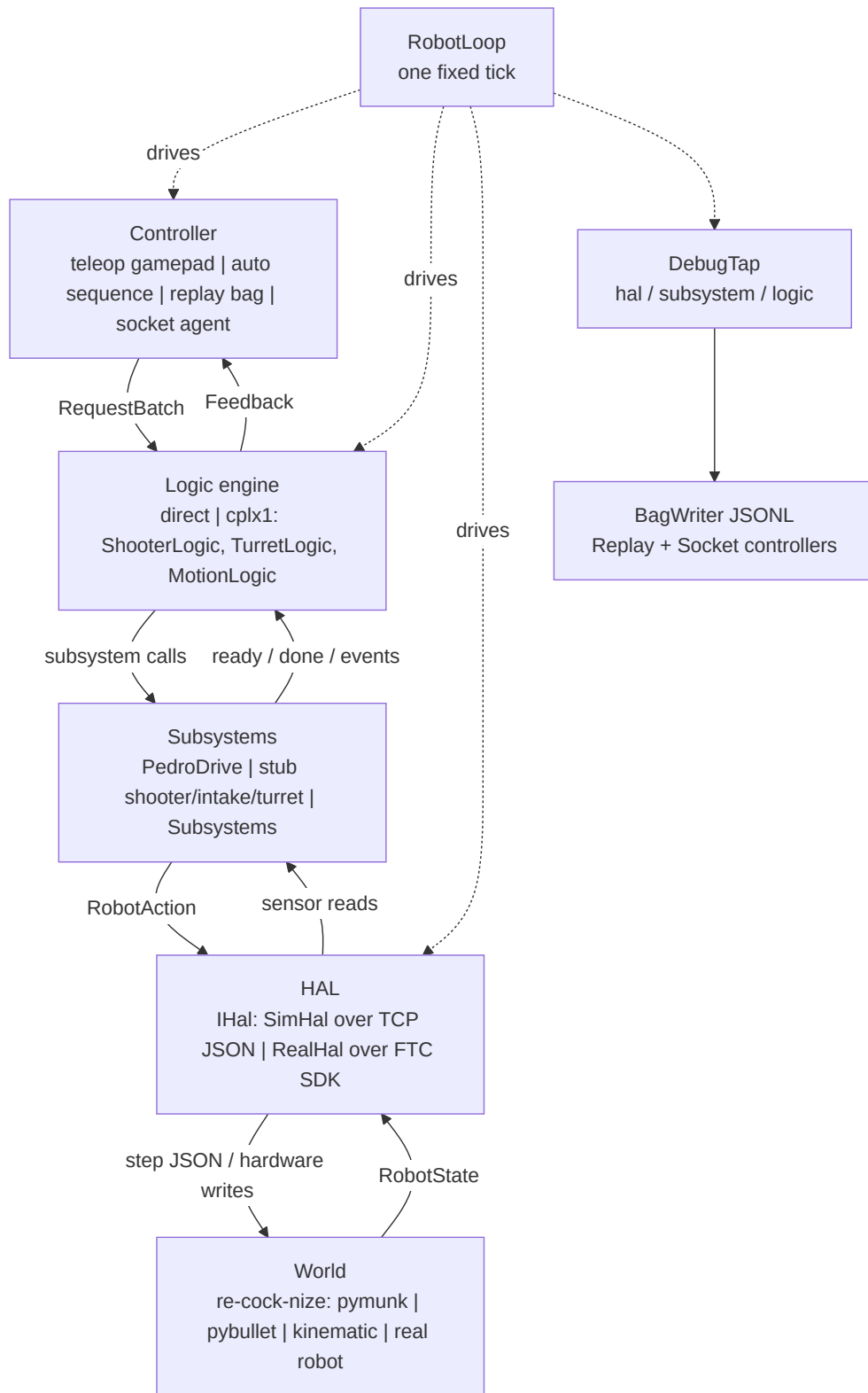


Figure 1: Diagram 1

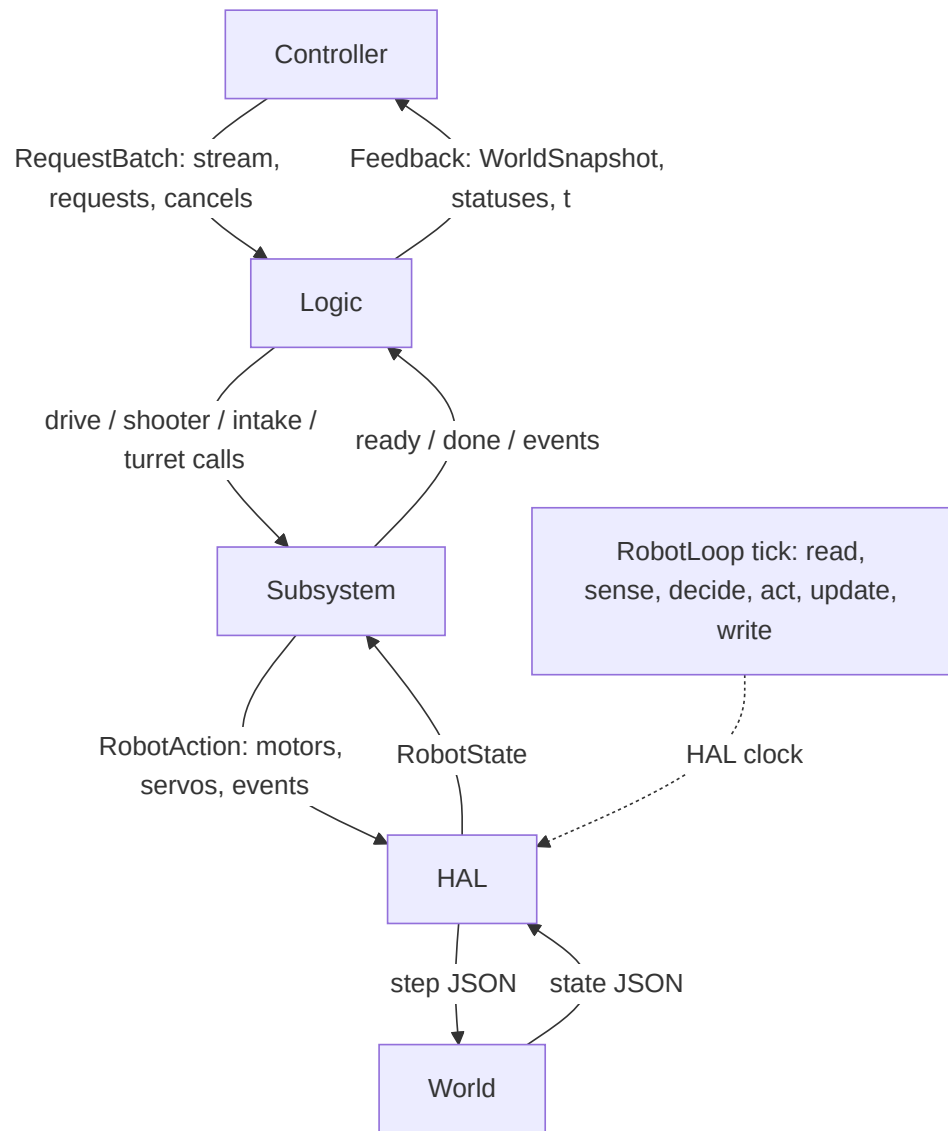


Figure 2: Diagram 2

Variant B – Layer handbook

The handbook follows the runtime stack from the controller down to the world. Solid arrows are calls or records; dotted arrows are orchestration rails. Sources are the current `robot-code dev-phase-1.1` head `dc4da66` and `re-cock-nize dev-phase-1.1` head `518bf9c`.

1. Controller

Method	Arguments	Returns	Caller / consumer
<code>IController.decide</code>	<code>Feedback(world, statuses, t)</code>	<code>RequestBatch(stream, requests, cancels)</code>	<code>RobotLoop</code> ; engine consumes the batch
<code>TeleopController.decide</code>	<code>HAL GamepadState, prior Feedback</code>	mapped batch, optionally merged with <code>SequenceRunner</code>	live teleop
<code>AutoController.decide</code>	<code>Feedback</code>	next immutable auto batch	FTC autonomous opmode
<code>ReplayController.decide</code>	ignored feedback	next logic batch from JSONL, then idle	bag replay
<code>SocketController.decide</code>	<code>Feedback</code>	latest valid batch; timeout emits <code>CANCEL_ALL</code> once	external agent

- `Buttons` is controller-only edge and hold state; it never references a device.
- `TeleopMap` maps sticks to `RequestStream`; RB emits `SHOOT(3)`, LB toggles intake.
- Y starts a short `AutoSequence`; BACK emits compile-time `RESET_POSE`.
- START held for one second emits one `SWITCH_ENGINE` request.
- Manual stick input cancels the sequence-owned IDs before returning the batch.
- Controllers must not import `IHal`, `PedroDrive`, physics, or FTC classes.

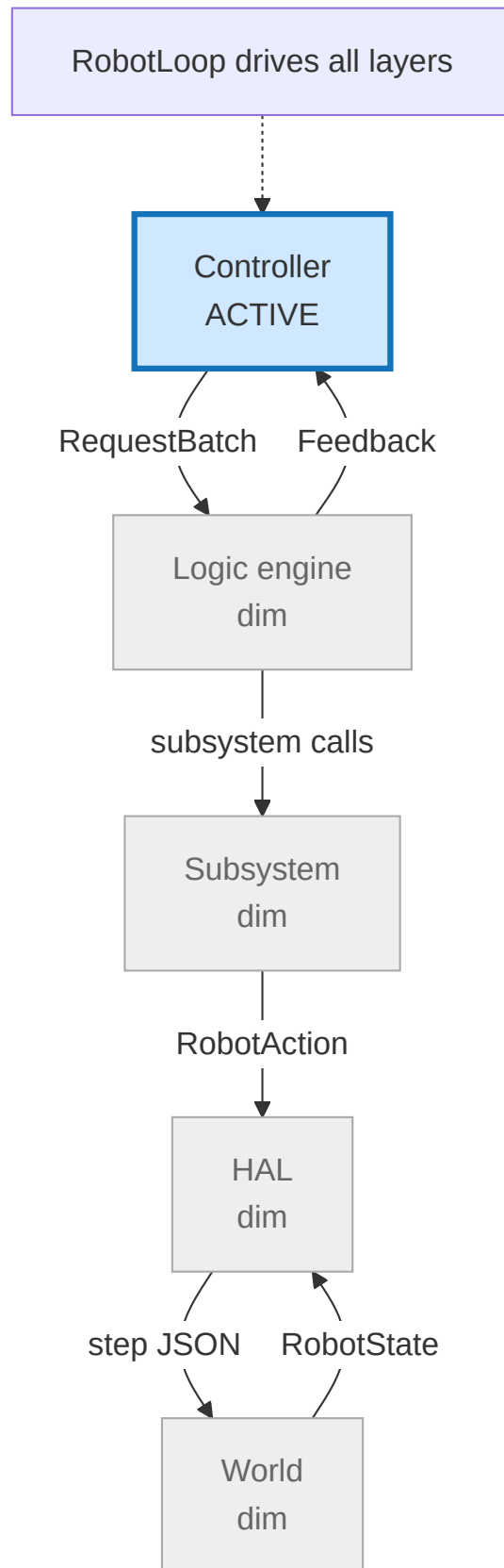


Figure 3: Diagram 3

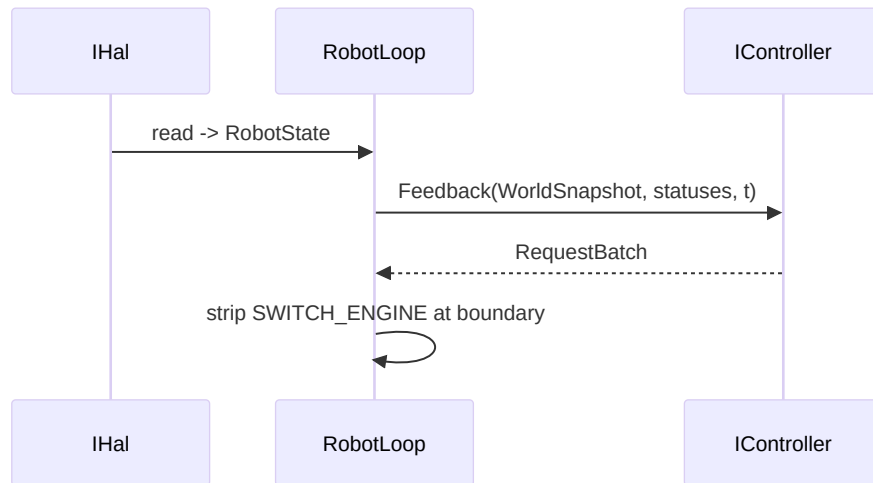


Figure 4: Diagram 4

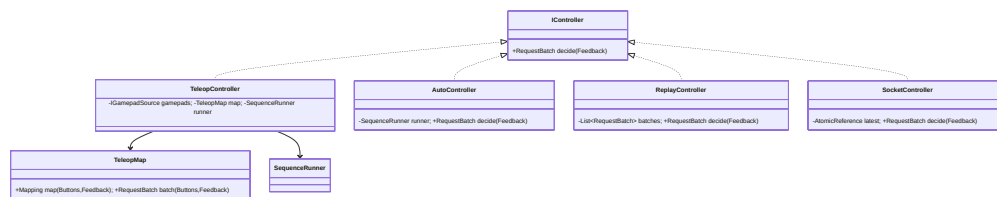


Figure 5: Diagram 5

2. Logic engine

Method	Arguments	Returns / side effect	Caller / consumer
<code>IRobotEngine.name</code>	none	direct or cplx1	RobotFactory, bags
<code>IRobotEngine.sense</code>	RobotState	WorldSnapshot; calls <code>Subsystems.observe</code>	RobotLoop
<code>IRobotEngine.act</code>	RequestBatch	updates jobs and subsystem commands	RobotLoop
<code>IRobotEngine.action</code>	none	latest RobotAction	RobotLoop -> HAL
<code>IRobotEngine.drainStatuses</code>	sense	statuses from preceding act	RobotLoop -> controller next tick
<code>DirectEngine</code>	Subsystems + DirectMap	stream arbitration and edge jobs	RobotFactory
<code>CplxEngine1</code>	Subsystems + three logic modules	automatic aim, motion, shooter sequencing	RobotFactory

- `DirectEngine` owns one drive job and one shooter job; `DirectMap` rejects busy work.
- `CplxEngine1` owns `MotionLogic`, `TurretLogic`, `ShooterLogic`, and intake ownership.
- `MotionLogic` arbitrates manual stream against `GOTO`, `PATH`, and `TURN_TO`.
- `TurretLogic` aims automatically at the alliance goal and holds during feeding.
- `ShooterLogic.State` is `IDLE`, `SPINNING`, `FEEDING`, or `DONE`.
- Individual shooter cancel IDs are honored by cplx1; `CANCEL_ALL` is always honored.
- `RESET_POSE` reaches cplx1's handler through `MotionLogic`; `RobotLoop` owns switching.

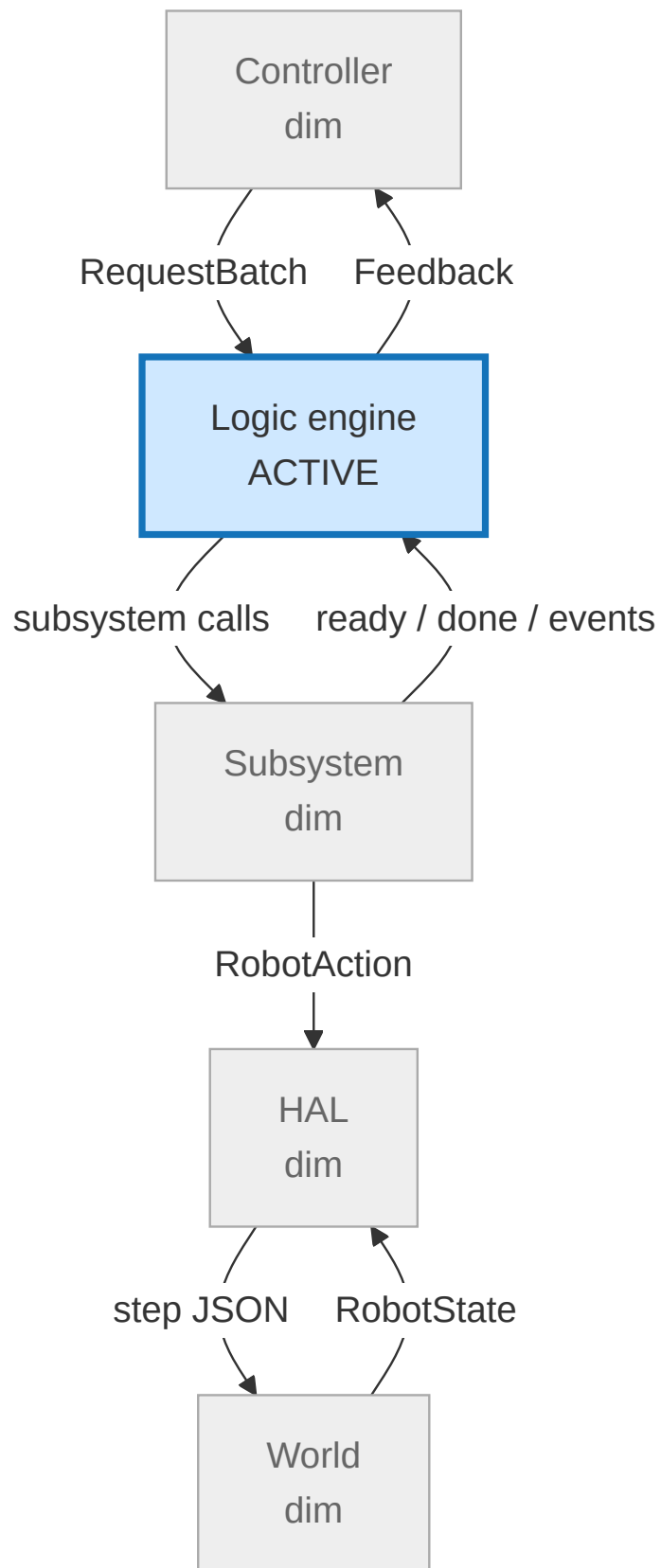


Figure 6: Diagram 6

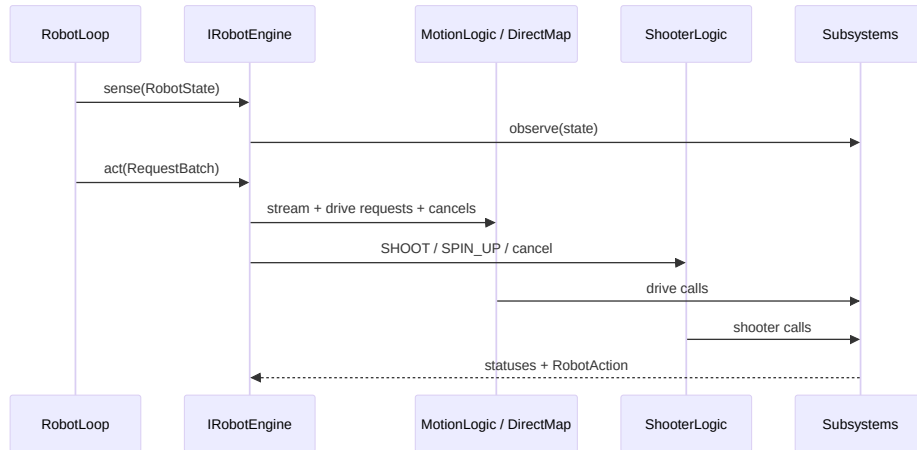


Figure 7: Diagram 7

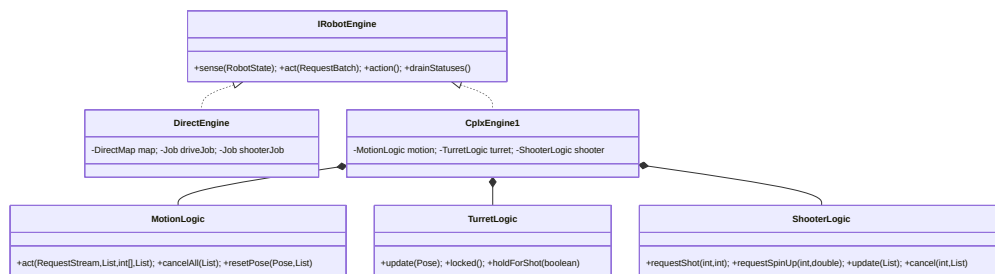


Figure 8: Diagram 8

3. Subsystem

Interface	Methods	Implementation(s)	Owner above the seam
ISubsystem	observe(RobotState), update(RobotAction.Builder)	all mechanisms	Subsystems fixed order
IDrive	manual, follow, turnTo, stop, resetPose, pathDone, pose	PedroDrive	DirectMap / MotionLogic
IShooter	spinUp, spinDown, isReady, feed, isFeeding	StubShooter	DirectMap / ShooterLogic
IIntake	run, stop, hasBall	StubIntake	engines
ITurret	aimAt, scan, hold, onTarget, angleRad	StubTurret	DirectMap / TurretLogic
Subsystems	observe, update	record of four interfaces	both engines

- Subsystems translate stable motor/servo physics into a narrow mechanism API.
- `PedroDrive` bridges `HalDrivetrain` and `HalLocalizer`; it is the only Pedro adapter.
- Stubs are timing/state machines, not physical shooter, intake, or turret models.
- `SubsystemTrace` can wrap all four and records calls for `DebugTap`.
- A subsystem must not inspect gamepad edges, request IDs, or JSON.
- `RobotAction` is assembled once per tick after every `observe` call.

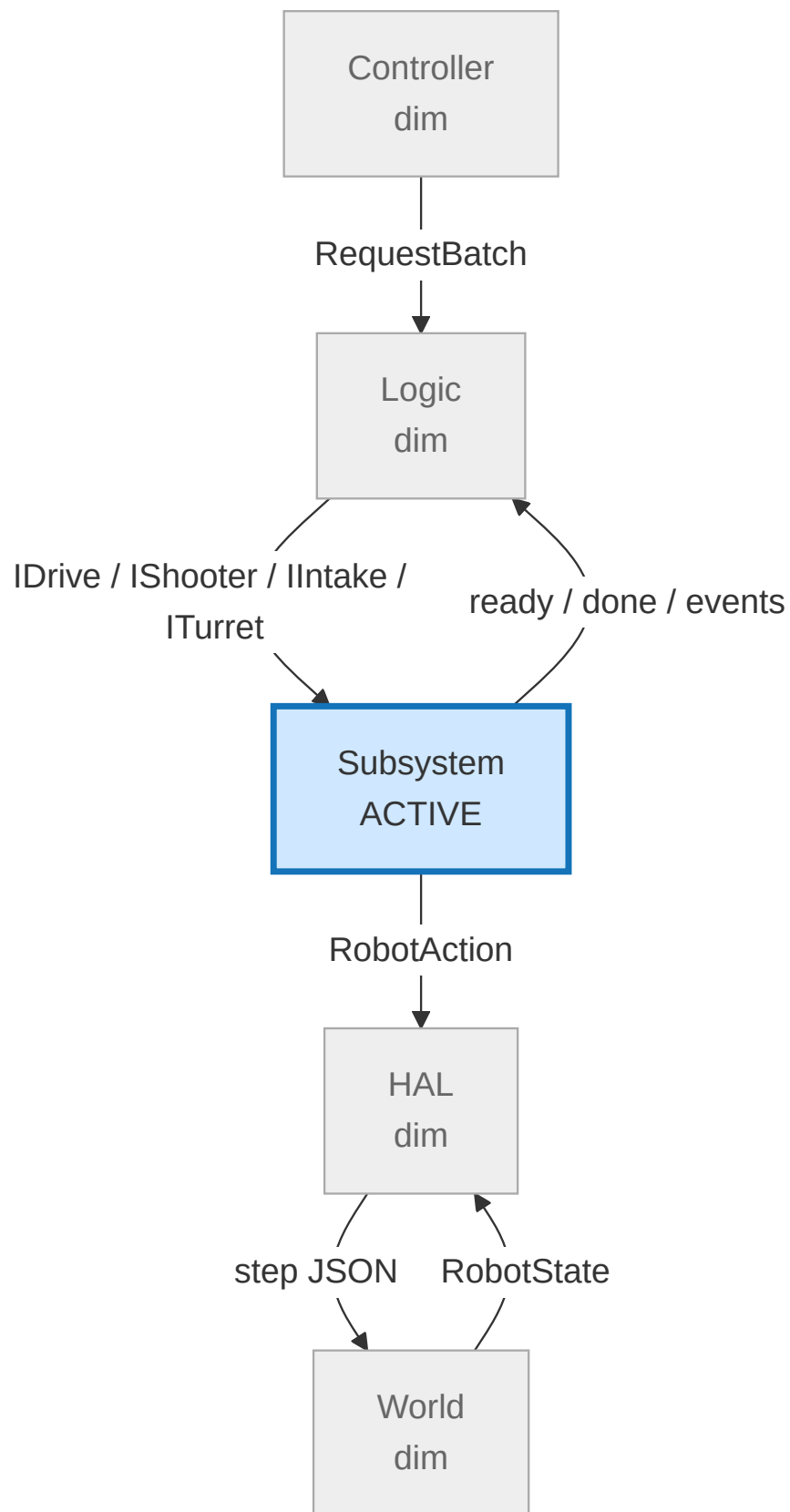


Figure 9: Diagram 9

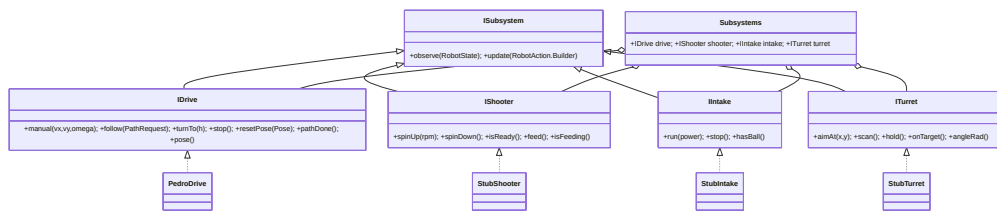


Figure 10: Diagram 10

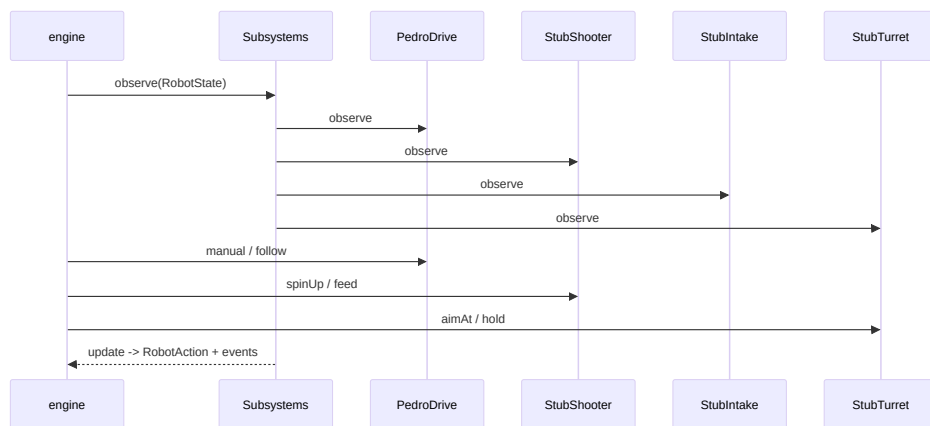


Figure 11: Diagram 11

4. HAL

Method	Arguments	Returns	Implementation
<code>IHal.now</code>	none	HAL-clock milliseconds	<code>RealHal</code> , <code>SimHal</code>
<code>IHal.read</code>	none	<code>RobotState</code> (enc, vel, yaw, Pinpoint, voltage)	same
<code>IHal.write</code>	<code>RobotAction</code> motor/servo maps and events	<code>void</code>	same
<code>IGamepadSource.get</code>	none	latest <code>GamepadState</code>	same
<code>SimHal</code> handshake	<code>seed</code> , <code>Pose</code>	<code>ready</code> plus state at <code>t_ms=0</code>	Java <code>:sim</code> <-> Python server

- HAL owns time, device I/O, gamepad sampling, and no policy.
- `RobotState.t` comes from HAL; `RobotState` never contains simulator truth.
- `RobotAction` keys are compile-time `RobotConstants` names; missing motor means zero.
- `SimHal` sends one line per step and blocks for its matching state.
- `RealHal` is in the FTC/Android module; `:core` has no SDK imports.
- `RobotConstants.java` is read by the Python simulator with a small regex parser.

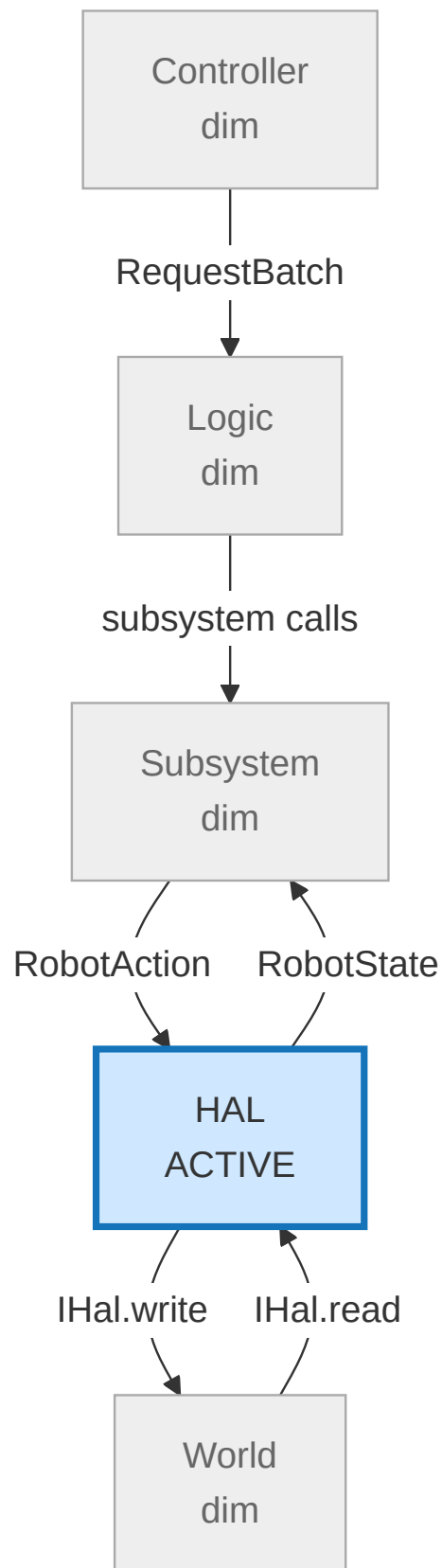


Figure 12: Diagram 12

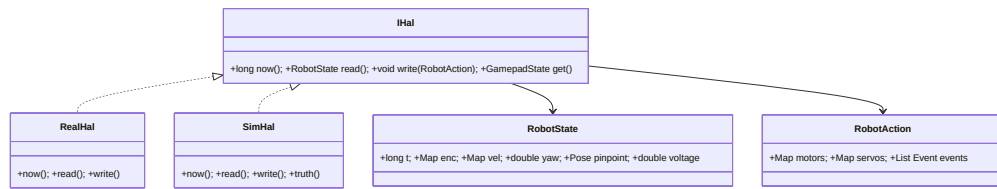


Figure 13: Diagram 13

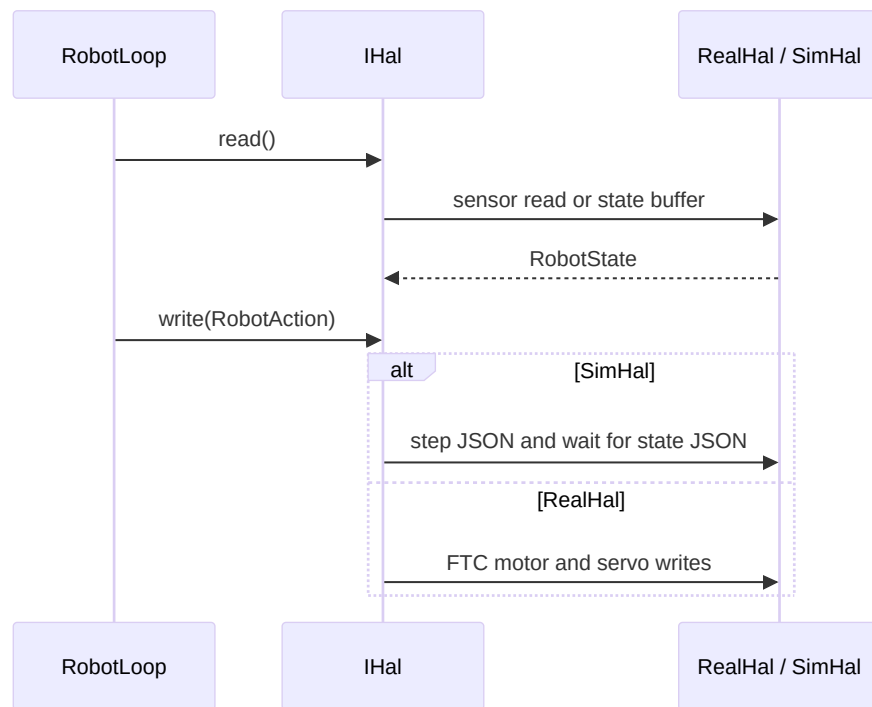


Figure 14: Diagram 14

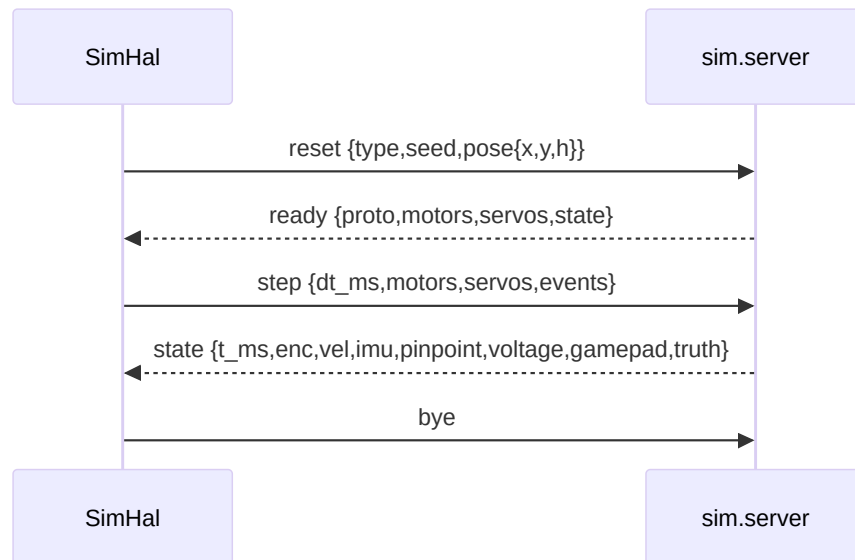


Figure 15: Diagram 15

5. World and simulator

Component	Owns	Current behavior
<code>sim.server</code>	TCP protocol, reset barrier, viewer	line-delimited JSON, protocol 1
<code>PhysicsBackend</code> <code>MotorModel</code>	body integration and contacts shared motor lag, encoders, mecanum mapping, noise	<code>pymunk</code> , <code>pybullet</code> , or <code>kinematic</code> first-order RPM and seeded sensors
<code>MultiRobotWorld</code>	one backend per robot, shared world where supported	lockstep <code>--robots N</code>
viewer	pygame field display and gamepad events	omitted by <code>--headless</code>

- Java owns `dt_ms`; simulation can run faster than wall time in headless mode.
- `Pymunk` and `PyBullet` share one world in multi-robot mode; kinematic bodies step independently.
- `truth` is for tests/viewer only and is never copied into `RobotState`.
- Python reads motor, wheel, battery, Pinpoint, and stub timing constants from Java.
- Servo commands are validated but non-empty servo physics is intentionally unsupported.
- Deterministic reset seed and ordered lockstep submissions are part of protocol version 1.

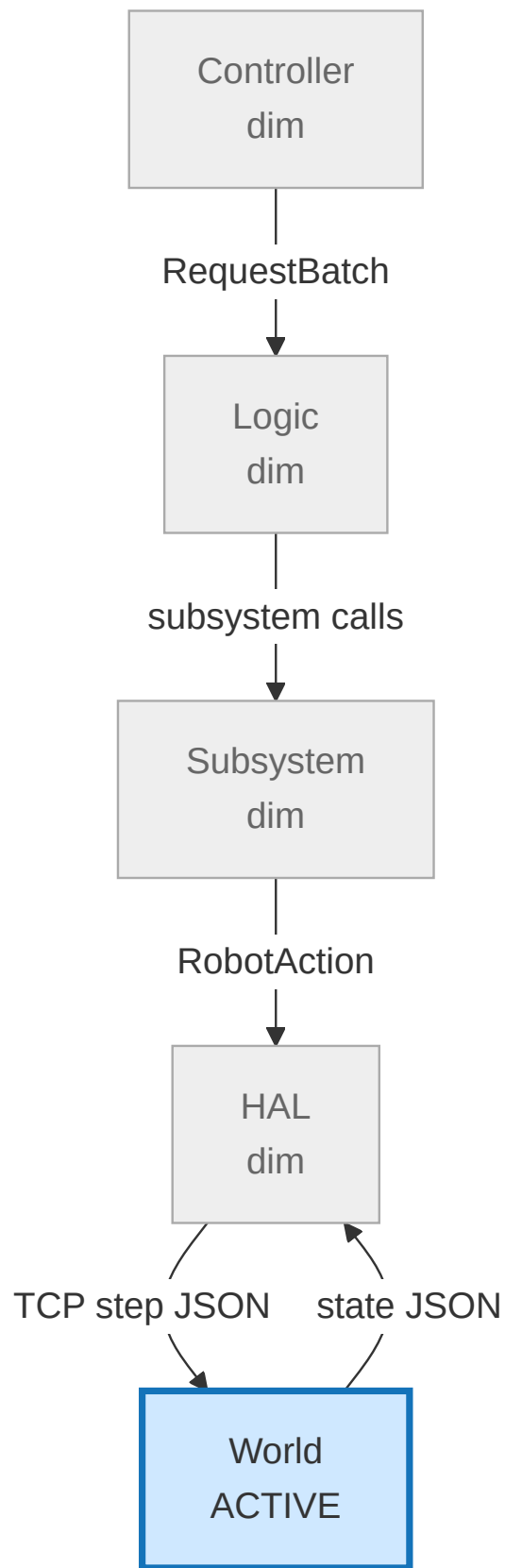


Figure 16: Diagram 16

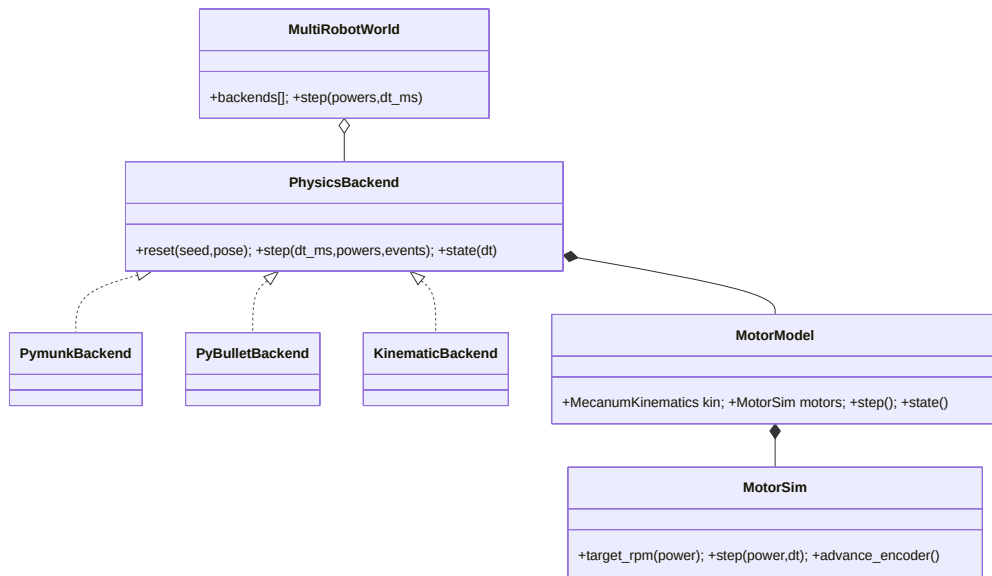


Figure 17: Diagram 17

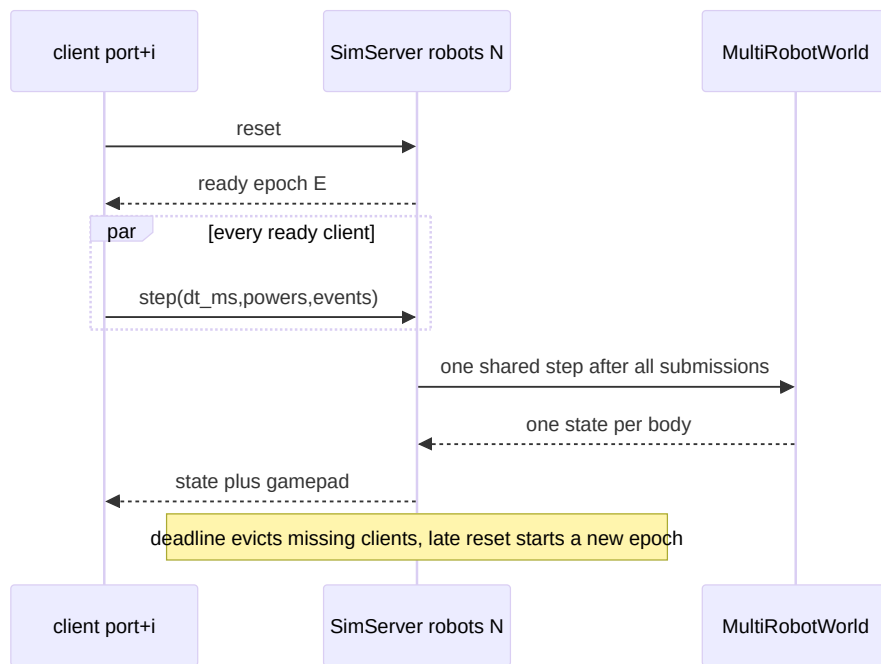


Figure 18: Diagram 18

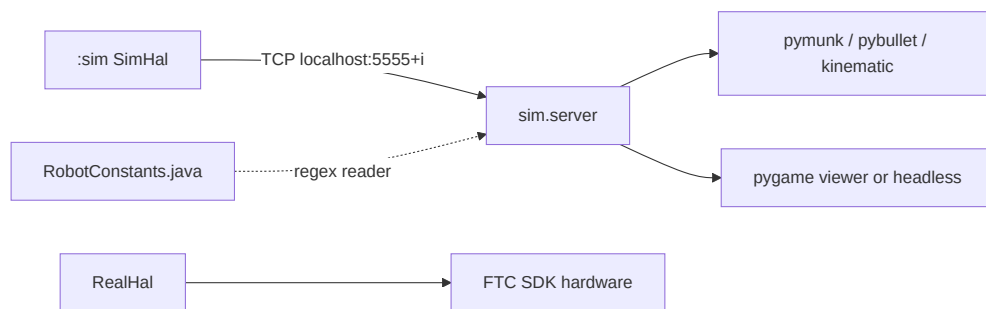


Figure 19: Diagram 19

6. Cross-layer records and seams

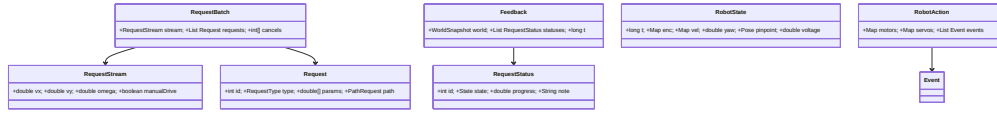


Figure 20: Diagram 20

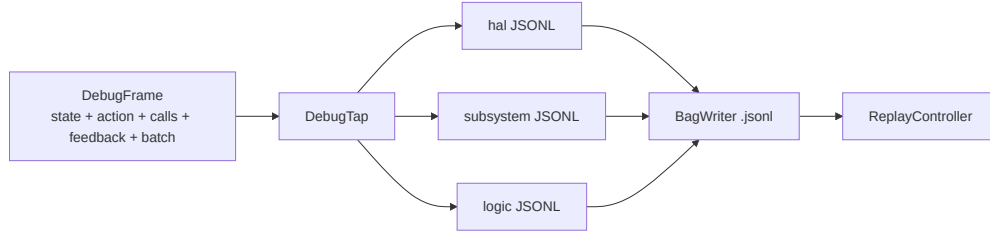


Figure 21: Diagram 21

- Records copy their collections; only the owning layer mutates its private jobs.
- **RequestStatus** terminals are **DONE**, **FAILED**, and **REJECTED**; feedback lags one tick.
- **DebugTap** receives immutable frames through a bounded global queue (capacity 512).
- The dispatcher writes the bag before offering each seam line to bounded clients.
- Socket and replay controllers sit above the same **IController** seam as teleop and auto.

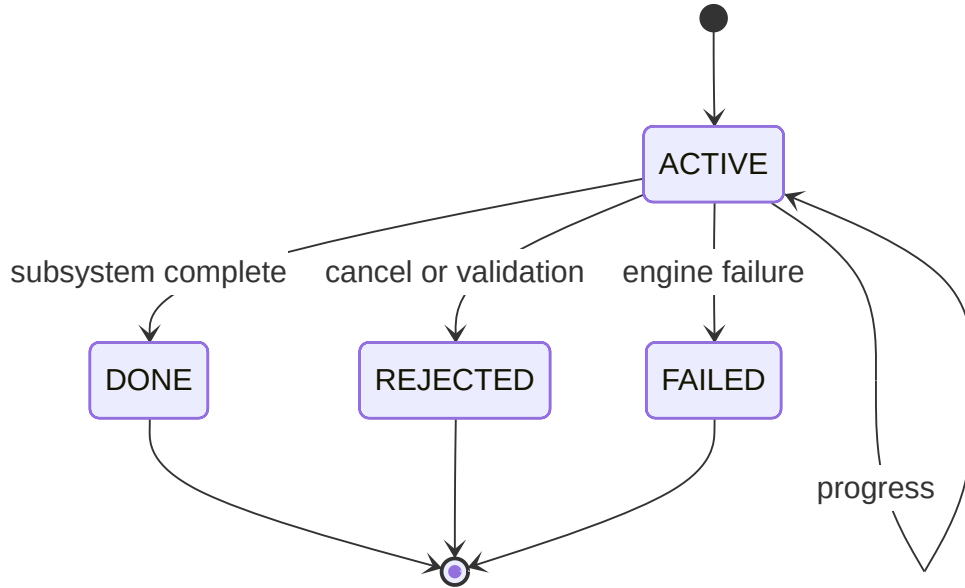


Figure 22: Diagram 22